User Guide

Delphi App Translation Studio

Version 1.0

Last changed: August 7, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

Table of Contents

1. Welcome

Delphi App Translation Studio is an offline-first Windows developer tool for adding localization to existing Delphi VCL and FireMonkey applications. It scans designer-authored forms and Delphi resourcestring declarations, stores the results in an editable development catalog, coordinates automatic in-place translation by Codex or Claude, verifies protected data, validates translation safety, and exports a compact JSON pack for the target application.

The Studio itself is built with FireMonkey, but it works with both VCL and FMX target projects. The supported build targets are Windows Win32 and Win64. macOS, iOS, Android, Linux, C++Builder, and runtime cloud translation are outside the product scope.

> **Offline by design.** Codex or Claude can edit the saved development catalog directly in the developer workspace; no CSV round trip or translation-provider key is required. Google and DeepL remain direct-provider alternatives. A translated target application reads local JSON and never needs Internet access or credentials.

1.1 What the Studio changes

- Scanning is read-only and does not alter the selected project.
- Saving creates project-local Localization\Development catalog files.
- Export creates project-local Localization\Languages runtime packs.
- Integration is a separate explicit operation with preview, verified backup, apply, and restore.
- Language menu items are persisted in the target DFM or FMX so they remain editable in the Delphi IDE.

2. Installation and Startup

The open-source project can be opened and compiled by anyone with Delphi 13 / RAD Studio 13 Florence. No installer is required for development builds. Open DelphiAppTranslationStudio.dproj, choose Win32 or Win64, and build.

Build products are placed under bin\<Platform>\<Configuration>. Compiler units are placed under dcu. The project deliberately does not target mobile or macOS platforms.

2.1 First launch

1. Run DelphiAppTranslationStudio.exe.
2. Confirm that the title reads Delphi App Translation Studio.

3. The Studio opens maximized. Use the left workflow panel; every page expands into the available workspace and the blue selection bar follows Project, Scan, Translate, Validation, Export, Integration, and Engine Settings.

3. The End-to-End Workflow

Step	Purpose	Primary output
1 Project	Open and identify a Delphi project.	Project profile
2 Scan	Extract designated designer text and resourcestrings.	Scan result
3 Translate	Select a language and run automatic in-place agent translation.	Protected AI draft catalog
4 Validation	Check completeness and structural safety.	Issue list
5 Export	Create the compact offline pack.	Runtime JSON
6 Integration	Preview/apply runtime and language-menu wiring.	Integrated target project
7 Engine Settings	Configure Codex CLI or Claude Code; optionally configure Google or DeepL.	Automatic agent or direct-provider drafts

4. Open and Scan a Project

1. Choose Project and select Open Delphi Project.
2. Open a .dproj when available; a .dpr is also accepted.
3. Review the detected framework, platforms, form count, and source count.
4. Choose Scan. The Studio reads text DFM/FMX resources and resourcestring declarations.
5. Review the entry count, elapsed milliseconds, breakdown, and result list.

Binary forms. The scanner expects text DFM/FMX resources. If a legacy DFM is binary, use Delphi's normal form conversion facilities before scanning and keep a source-control copy.

4.1 Text that is collected

- Captions, Text values, prompts, hints, headings, and designated list content.
- Memo and string-list content when it represents user-visible text.
- Delphi resourcestring symbols with stable unit-and-symbol keys.
- Locale metadata such as date, time, number, and currency formatting is entered on the Languages page rather than guessed from UI text.

5. Create and Edit a Language Catalog

1. Choose Translate after scanning.
2. Select the source language, normally English (United States) [en-US].
3. Select the target language from the built-in language list. The Studio records its locale code and native name.
4. Review the automatically selected left-to-right or right-to-left direction.
5. Review or edit the date, time, decimal, thousands, and currency fields. Blank fields are initially populated from Delphi TFormatSettings for the target locale.
6. Choose Save.

The catalog is saved under Localization\Development using the project name and target locale. Each target language has its own development catalog. Existing translations are preserved on later scans when the stable key and source text remain unchanged. Changed source text is flagged for review, and removed entries become obsolete rather than disappearing silently.

5.1 Translation statuses

Status	Meaning
Needs translation	No usable target text is present.
AI draft	Codex or Claude produced an in-place draft that passed protected reload.
Machine translated	DeepL or Google produced a draft that requires review.
Imported	CSV supplied target text that requires review.
Edited	A person changed or entered the target text.
Reviewed	A person reviewed the target text.

Status	Meaning
Approved	The entry is release-approved.
Source changed	The source changed after a translation existed.
Excluded	The entry is intentionally omitted.
Obsolete	The source entry no longer exists in the latest scan.
Error	The entry needs correction after a failed operation or check.

5.2 Automatic in-place Codex or Claude workflow

1. Open Engine Settings and select Codex CLI or Claude Code.
2. Install and sign in to that command-line agent using its normal vendor instructions. Browse to the executable or choose Detect, then choose Check Installation. The Studio stores only the executable path, engine, and model choice - never the agent's credentials.
3. Return to Translate, select the target language, and choose Translate Automatically.
4. The Studio saves the catalog, creates an exact recovery snapshot and terminology profile, starts the signed-in agent in the target project directory, supplies the protected translation contract through standard input, and monitors the process and catalog.
5. When the agent exits successfully, the Studio verifies every protected field, adopts only allowed translation changes as AI Draft, saves the canonical JSON, removes the recovery snapshot, refreshes the page, and runs validation.
6. If the process fails or Cancel Translation is chosen, the Studio terminates only that child process and restores the exact pre-translation catalog. Prompt and Reload remain recovery/diagnostic actions, not required normal steps.

> **No interchange detour.** The installed agent edits the canonical project-local JSON itself. CSV remains available for translation companies and spreadsheet workflows, but it is not required for Codex or Claude. The automatic path uses the developer's existing agent sign-in rather than a translation-provider API key.

5.3 Safety, context, and focused review

- Designer-property entries are applied automatically by the VCL or FMX runtime adapter.
- Pascal resourcestring entries require an explicit TranslateText(Key, Fallback) call at the intended code location. Mark Manual wiring confirmed only after adding and reviewing that call.
- Translation origin is independent of Draft, Reviewed, and Approved status. A valid AI result is never silently approved.

- Only Needs Translation, Source Changed, Error, and existing AI Draft entries are eligible. Machine-translated, Imported, Edited, Reviewed, Approved, Excluded, and Obsolete work is protected.
- A metadata-only AI confirmation may retain an existing translation that remains correct after a source change; changed entries must identify Codex or Claude and record high, medium, or low confidence.
- The terminology profile records application context, audience, tone, formality, protected names, and preferred terminology.
- Validation focuses on placeholders, accelerators, low confidence, explicit AI review notes, inconsistent repeated terms, source changes, and runtime wiring rather than listing every structurally valid AI draft.
- Exact-source translations from other stable keys are suggestions only. The developer must explicitly accept one, and the accepted target remains Edited rather than inheriting approval.

6. Engine Settings and Provider Alternative

The primary Engine Settings controls select an installed Codex CLI or Claude Code executable and the default model. Detect searches supported command-line locations; Browse accepts an explicit .exe, .cmd, or .bat; Check Installation runs the agent's version command. The developer signs in using the agent's own supported login flow before starting translation.

When a direct provider is preferred, the same page also supports DeepL API Free, DeepL API Pro, and Google Cloud Translation Basic v2. Provider accounts, billing, quotas, prices, and supported languages are controlled by the provider and may change. Codex/Claude automatic translation does not use these API-key settings.

Control	Behavior
Provider	Select DeepL or Google Cloud Translation.
DeepL API plan	Select API Free or API Pro so the Studio uses the matching endpoint.
API key	Masked field for a new or replacement key.
Remember securely	Stores the key as a Windows Generic Credential.
Session only	Clear Remember before saving; the key is held only until shutdown.
Replace / Save Key	Records the entered key using the selected storage choice.

Control	Behavior
Test Connection	Sends a small English-to-Italian request.
Remove Key	Deletes stored and session copies for the selected provider.
Timeout / batch	Controls 5-300 second requests and 1-50 strings per request.

7. Obtain and Enter a DeepL API Key

> **Use a DeepL API account.** A normal DeepL Translator subscription is not automatically a DeepL API plan. Confirm that the account includes API Free or API Pro.

1. Open the official DeepL developer site at https://developers.deepl.com/ and review current API account requirements.
2. Create or sign in to a DeepL account and subscribe to DeepL API Free or DeepL API Pro as appropriate.
3. Open the account's API Keys tab. Create a separate key for this Studio when the account interface permits multiple keys.
4. Copy the key once and keep it out of source code, JSON catalogs, issue trackers, email, and screenshots.
5. In the Studio, choose Engine Settings and select DeepL in the optional provider area.
6. Select API Free or API Pro to match the account. Free uses api-free.deepl.com; Pro uses api.deepl.com.
7. Paste the key into the masked field.
8. Leave Remember securely checked for Windows Credential Manager, or clear it for Use for This Session Only.
9. Choose Replace / Save Key, then Test Connection.
10. If the test fails, verify the plan selection, key status, account quota/billing, firewall, and target-language availability.

DeepL key and authentication reference: https://developers.deepl.com/docs/getting-started/auth

DeepL quickstart: https://developers.deepl.com/docs/getting-started/quickstart

DeepL multiple-key guidance: https://developers.deepl.com/docs/multiple-api-keys

8. Obtain and Enter a Google API Key

> **Google Basic v2 only.** The Studio uses Cloud Translation Basic v2 because it supports API-key authentication. Cloud Translation Advanced v3 uses different authentication and is not implemented.

1. Open Google Cloud Console at https://console.cloud.google.com/ and sign in.
2. Create a dedicated Google Cloud project or select an existing project intended for this Studio.
3. Attach an active billing account if Google requires billing for the Cloud Translation API.
4. Open APIs & Services, then Library. Find and enable Cloud Translation API.
5. Open APIs & Services, then Credentials. Choose Create credentials and API key. Current Google Cloud policy requires at least one API restriction when creating a key.
6. Under API restrictions, restrict the key to Cloud Translation API. Apply any additional application restriction that is compatible with the developer computer and organization; desktop usage commonly makes website-referrer restrictions unsuitable.
7. Review quotas and budget alerts in Google Cloud before bulk translation.
8. In the Studio, choose Engine Settings and select Google Cloud Translation in the optional provider area.
9. Paste the key into the masked field. The DeepL plan control is disabled because it does not apply.
10. Choose persistent Windows Credential Manager storage or session-only use, then Replace / Save Key and Test Connection.

Google Cloud Translation setup: https://docs.cloud.google.com/translate/docs/setup

Google API-key creation and management: https://docs.cloud.google.com/docs/authentication/api-keys

Google API-key restrictions: https://docs.cloud.google.com/api-keys/docs/add-restrictions-api-keys

Cloud Translation authentication: https://docs.cloud.google.com/translate/docs/authentication

9. Direct Provider Translation

1. Open or create the target-language catalog.
2. Confirm the source and target language codes.
3. Configure and test a provider under Engine Settings.
4. Choose Google / DeepL Translation on the Translate page.
5. Read the confirmation message showing how many unresolved strings will be sent to the provider. Cross-key suggestions are never accepted automatically.
6. Choose Yes to begin. Existing complete reviewed or approved work is preserved.
7. Wait for all batches. Transient HTTP 429 and provider server failures receive bounded retries.
8. Review every machine-translated entry for meaning, placeholders, accelerators, product terminology, tone, and available control space.

9. Save the catalog and run Validation.

> **Cost control.** The confirmation count is not a price quote. Provider billing can depend on characters and account terms. Check the provider dashboard and quotas before a large request.

10. Validate and Export

Structural validation checks catalog metadata, required translations, duplicate keys, source changes, Delphi indexed/sequential placeholders, accelerator keys, and excluded/obsolete conditions. Errors block export. Manual resourcestring wiring is reported as a runtime-readiness warning. Linguistic Reviewed and Approved states are reported separately and are not inferred from structural validation.

1. Choose Validation and Run Validation.
2. Open each listed issue and correct the catalog entry or metadata.
3. Repeat until no errors remain.
4. Choose Export and Export Runtime Pack.
5. Record the output path under Localization\Languages.

11. Integrate a Target Application

Integration adds the offline runtime, generated application-specific unit, startup calls, event wiring, and designer-persisted language menu items. It does not add provider access to the target.

1. Close or save the target project in Delphi.
2. Select Integration and confirm the designated language menu component, normally mnuLanguage.
3. Choose Build Integration Plan and review each proposed operation.
4. Choose Generate Preview. Select every changed file and read its line-numbered original/proposed text. Newly generated files are shown in full.
5. After every file has been viewed, check I reviewed every exact change. Apply remains disabled until that confirmation is checked.
6. Choose Apply. The Studio creates and verifies a pre-change backup, writes atomically, and records a restore manifest.
7. Reopen the target project, inspect the DFM/FMX menu items in the IDE, and build Win32 and Win64.
8. Run the target, select a language, restart if required by the application's integration pattern, and verify all forms.
9. Use Restore to recover the recorded pre-integration state if needed.

11.1 Runtime files and preferences

- Deploy JSON packs beside the target executable under Localization\Languages.
- Run Deploy-LanguagePacks.ps1 from the preview package with -ApplicationDirectory pointing to the output folder, or copy the files through the Delphi deployment manager.
- The selected language is stored in %LOCALAPPDATA%\<ApplicationId>\language.ini.
- The executable folder can therefore remain read-only, as it normally is under Program Files.
- VCL startup applies the first form through generated DPR wiring. FMX integration persists an OnCreate handler in the form resource and applies translation after FMX streaming; an existing handler is preserved.

12. Translate the Studio Itself

1. Run the English Studio and select DelphiAppTranslationStudio.dproj.
2. Scan it and create the desired language catalog.
3. Translate, review, validate, and export the pack.
4. Build the self-integration preview for mnuLanguage.
5. Apply the persisted FMX menu-resource update.
6. Close the running Studio, rebuild it, and start the newly built executable.
7. Select the new language. The preference is applied on the next launch.

The running executable is never overwritten. The JSON pack and preference are separate from the executable, and the rebuild supplies the updated menu on the next run.

13. Troubleshooting

Symptom	Likely action
Project framework unknown	Open the .dproj, confirm VCL/FMX units and project metadata, then rescan.
No scan entries	Confirm text DFM/FMX resources and designated user-visible properties.
HTTP 401/403	Replace the key; verify provider plan, API enablement, restrictions, and billing.
HTTP 429	Quota or rate limit reached; wait, reduce batch size, or review provider quota.

Symptom	Likely action
DeepL test fails only on one plan	Select API Free or API Pro to match the account endpoint.
Google test fails	Confirm Basic v2 Cloud Translation API is enabled and key API restriction permits it.
Export blocked	Run Validation and correct every error.
Language menu missing	Confirm the designated menu name and rebuild the integration plan.
Preference cannot be found	Check %LOCALAPPDATA%\<ApplicationId>\language.ini, not the executable folder.
Target remains English	Confirm the JSON pack applicationId and locale, deployment folder, selected preference, and startup ApplyTranslation calls.

14. Security and Privacy

- Only confirmed source strings are sent to the selected provider.
- Do not send secrets, personal data, customer data, or regulated information as UI text.
- Remembered keys are Windows Generic Credentials; session keys are not persisted.
- Removing a key affects only the selected provider.
- Provider settings JSON contains no API key.
- Catalogs, runtime packs, deployment packages, logs, backups, and Git should contain no key.
- Rotate a key immediately in the provider console if it may have been exposed, then replace it in the Studio.

15. File and Folder Reference

Location	Contents
Localization\Development	Editable full development catalogs.
Localization\Languages	Compact offline runtime JSON packs.

Location	Contents
export	Generated previews and temporary product output.
docs\guides / docs\pdf	Editable guides and companion PDFs.
%LOCALAPPDATA%\DelphiAppTranslationStudio	Studio settings and language preference, but no remembered key.
Windows Credential Manager	Remembered DeepL/Google Generic Credentials.
%LOCALAPPDATA%\<ApplicationId>	Integrated application's selected-language preference.

16. Provider Reference and Change Notice

Provider setup screens and commercial terms can change after this guide is published. Before creating a production key, compare these steps with the official pages listed in Chapters 7 and 8. Prefer a dedicated, restricted key and review the provider dashboard after the first bulk run.

This guide documents the implemented Studio as of August 7, 2026.